Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #6

Sensors – Signal Acquisition and Conditioning. Part 2: Analog Signal Acquisition

---

Author:
Alexandru Cebotari
Group FAF-233

Verified by:
Alexei Martiniuc

Chisinau, 2026

Technical task and scope

This report documents only Part 2 of the laboratory assignment. The implemented solution follows the required analog signal-conditioning pipeline and also satisfies Variant C by monitoring two temperature sensors at the same time: one analog and one digital.

The final application uses:

- one **NTC thermistor** on `A0` as the analog temperature channel;
- one **DHT22** on `D8` as the digital temperature channel;
- one **potentiometer** on `A1` for runtime acquisition-period control;
- one **LCD1602 in 4-bit mode** for simultaneous display of the two current temperatures and alert marks;
- two **LEDs** on `D12` and `D13` for per-sensor stable alerts;
- STDIO serial reporting for intermediate values, min/max values, trend direction, and runtime timing state.

The report intentionally ignores Part 1 and the control questions, exactly as requested.

Objectives

- Acquire one analog and one digital temperature signal periodically on Arduino Uno.
- Apply software conditioning required by Part 2: saturation, median filter, and weighted average.
- Convert the conditioned analog ADC signal into degrees Celsius.
- Display the two temperatures and alert states on an LCD, satisfying Variant C.
- Print all relevant intermediate stages, min/max values, trend direction, and timing state through STDIO.
- Allow runtime adjustment of acquisition and reporting period through a potentiometer.
- Keep the project modular, PlatformIO-compatible, and ready for Wokwi simulation.

1 1. Domain analysis

1.1 1.1 Technological context

Signal acquisition and conditioning in embedded systems is the process through which a physical phenomenon is transformed into a software-ready value that can be safely used for control, reporting, and supervision. In this laboratory, temperature is the measured quantity, but the emphasis is on the path from raw sensor output to a stable, conditioned value.

The analog channel illustrates the complete Part 2 pipeline. The thermistor produces a raw ADC value, the software applies saturation to constrain outliers, a median filter to suppress impulsive noise, and a weighted average to smooth small fluctuations while keeping the response fast enough for the laboratory latency requirement. The digital channel complements this with a second sensor on a digital interface, which is also conditioned and supervised so that Variant C is fully satisfied.

1.2 1.2 Selected components and technologies

- Hardware: Arduino Uno, Wokwi NTC thermistor, Wokwi DHT22, potentiometer, LCD1602 in 4-bit mode, two LEDs with 220 Ω resistors.
- Software tools: PlatformIO, Arduino framework, Wokwi, LaTeX, PlantUML.
- Libraries: DHT sensor library and LiquidCrystal.

The chosen hardware is intentionally simple and simulation-friendly. The thermistor provides the analog path required by Part 2, while the DHT22 provides a digital temperature source that can coexist on the same Arduino Uno and supports the two-sensor Variant C requirement.

1.3 1.3 Existing-solution reasoning and chosen variant

The assignment allows an analog or digital temperature sensor for Part 2, but the grading bonus for Variant C explicitly requests one analog and one digital sensor, with values and alerts displayed for both on an LCD. Therefore, the selected architecture is not the minimum solution; it is a full-score solution.

Compared with a single-sensor implementation, the dual-channel design is stronger for validation because it demonstrates both ADC-based acquisition and digital-interface acquisition under the same software architecture. It also proves that the filtering and reporting pipeline is reusable across multiple sensor types.

1.4 1.4 Problem definition

The application must periodically acquire temperature from an analog and a digital sensor, run the Part 2 conditioning stages, derive stable alert states, show the two tem-

peratures and alert marks on the LCD, and print a structured runtime report through STDIO. The report now also includes min/max values and trend direction, while a potentiometer allows runtime timing adjustment. The code must remain modular, reusable, and compatible with PlatformIO and Wokwi on Arduino Uno.

Section conclusion

The selected hardware and software stack directly matches Part 2, satisfies Variant C, and provides enough observability to document acquisition, conditioning, display, reporting, and validation in a methodical way.

2 2. Conceptualization and design

2.1 2.1 Technical requirements

ID	Requirement	Type	Acceptance target
R1	The application shall acquire the analog thermistor signal periodically.	F	New ADC samples every 50 ms
R2	The application shall acquire the digital DHT22 temperature periodically.	F	New digital readings available to the pipeline
R3	The analog path shall apply saturation before further conditioning.	F	ADC values constrained to a safe interval
R4	The analog path shall apply a median filter for salt-and-pepper noise rejection.	F	Impulsive outliers suppressed
R5	The application shall apply a weighted-average filter on conditioned samples.	F	Uniform noise reduced with limited delay
R6	The application shall convert conditioned analog ADC values into Celsius.	F	Valid engineering value produced
R7	The application shall report raw, saturated, median, weighted, and final values on STDIO.	F	Structured serial report visible
R8	The application shall display both current temperatures and alert marks on the LCD.	F	Variant C LCD display satisfied
R9	The application shall generate stable per-sensor alerts using threshold, hysteresis, and confirmation.	F	No unstable alert chatter
R10	The project shall remain modular, with dedicated drivers, services, and tasks.	NF	Separated files by responsibility
R11	The system shall respond to signal changes with latency under 100 ms.	NF	Nominal alert latency = 100 ms
R12	The solution shall build in PlatformIO and run in Wokwi with Arduino Uno.	NF	Successful build and simulation files present

Table 1: Requirements matrix

2.2 2.2 Validation criteria and traceability

Req.	Design anchor	Test IDs	Expected proof
R1	Analog thermistor driver + acquisition task	T1	ADC updates every acquisition cycle
R2	DHT22 driver + acquisition task	T2	Digital temperature updates correctly
R3	Saturation stage in sensor pipeline	T3	Extreme ADC values are clipped
R4	Median filter state and flow	T4	Short impulsive spikes are removed
R5	Weighted-average filter state and flow	T5	Smoothed output follows input without large oscillation
R6	`signal_conversion.cpp`	T6	Celsius value is plausible and stable
R7	Reporting task	T7	Serial monitor shows all intermediate stages
R8	LCD driver + display task	T8	Two temperatures and alert marks appear on LCD
R9	Threshold alert service	T9	Alerts remain stable near threshold
R10	Source tree organization	T10	Clear modular structure exists
R11	50 ms period, 2 confirmations	T11	100 ms nominal alert response
R12	PlatformIO + Wokwi files	T12	`pio run` succeeds and simulation files exist

Table 2: Requirement-to-test traceability

2.3 2.3 Architectural design

The implementation is split into four logical layers:

- **Application layer**: main controller and cooperative periodic scheduler.
- **Service layer**: conversion, saturation, filtering, shared signal pipeline, and threshold alert logic.
- **Driver layer**: thermistor acquisition, DHT22 access, LCD output, LEDs, and STDIO bridge.
- **Platform layer**: Arduino ADC, GPIO, timing, LCD, and Serial primitives.

This separation follows the documentation guideline directly because acquisition, conditioning, alerting, display, and reporting are isolated into reusable modules.

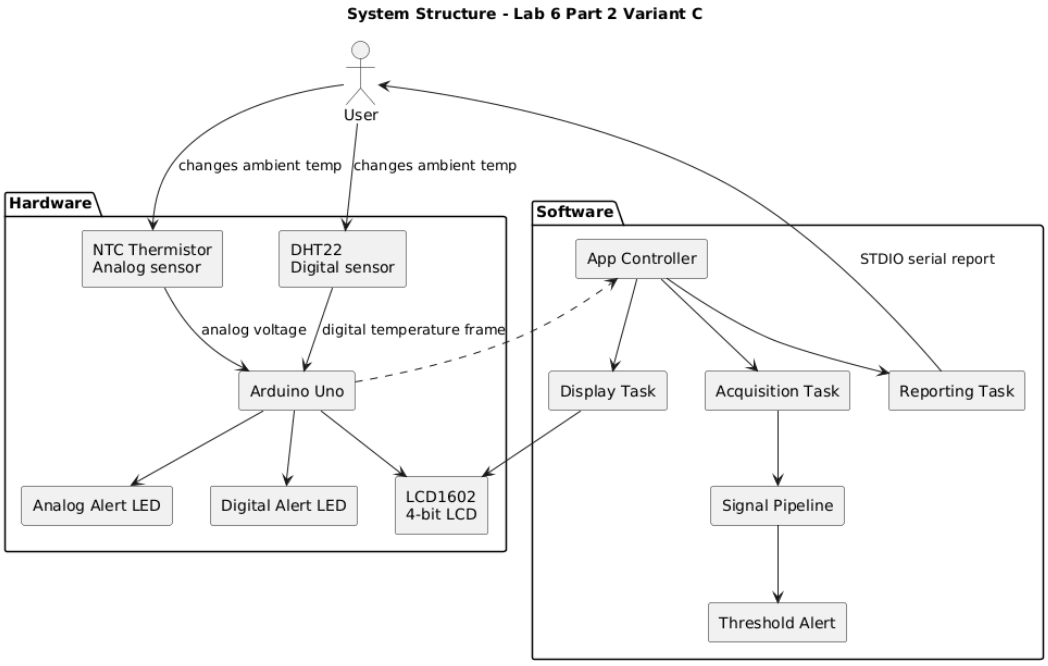

Figure 1: System structure

Figure 1 summarizes the full laboratory context from the user's interaction with the physical environment to the final embedded outputs. It makes clear that the system is not limited to raw acquisition: the two temperature sources are followed by software processing, alert evaluation, local visualization, and serial reporting. This figure is useful as a high-level map for the entire report because every later implementation decision can be traced back to one of these blocks.

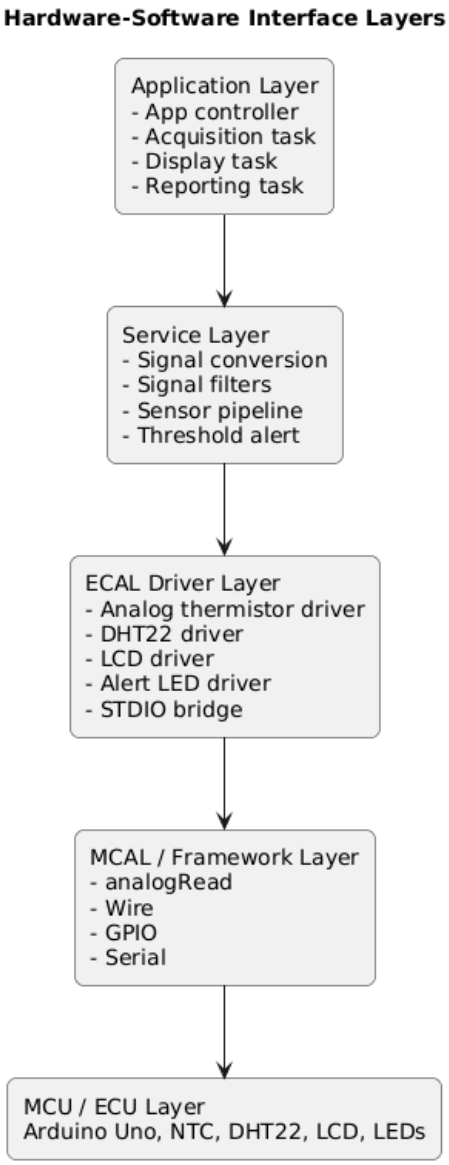

Figure 2: Hardware-software interface layers

Figure 2 highlights the layered separation adopted in the implementation. The application layer schedules work, the service layer contains reusable processing logic, and the driver layer isolates hardware access. This decomposition is important for maintainability because it allows filtering or reporting behavior to evolve without forcing changes in the low-level device interfaces.

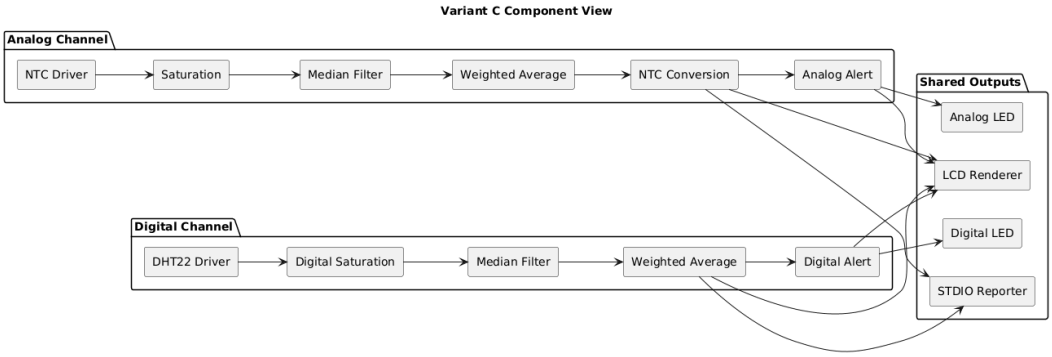

Figure 3: Variant C component view

Figure 3 explains how Variant C is satisfied in a concrete way. The analog and digital channels are modeled as parallel processing paths with shared output services, which makes the solution symmetrical and easier to validate. It also shows that the project is not just two independent sensors wired to the same board, but a coherent dual-channel monitoring architecture.

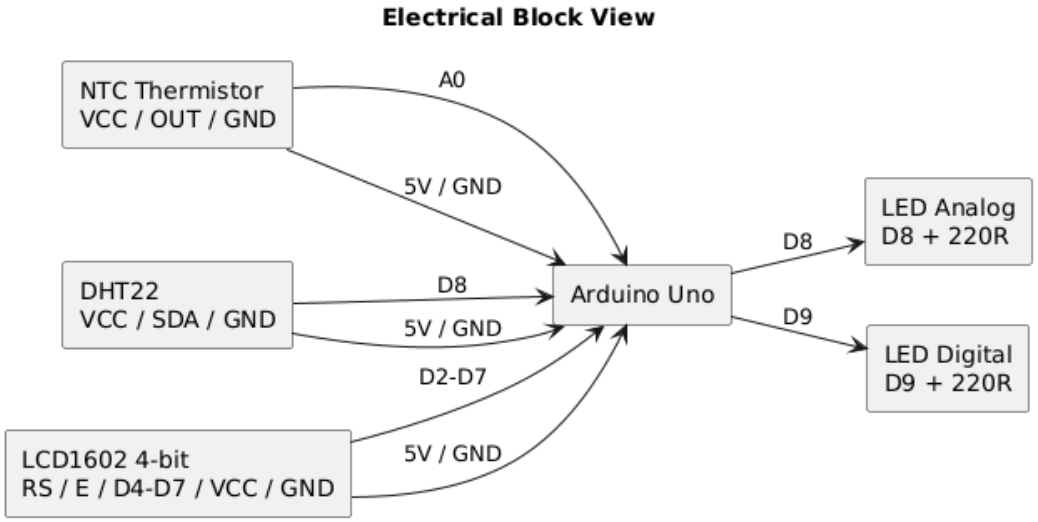

Figure 4: Electrical block view

Figure 4 focuses on the signal-level connections rather than software behavior. Its role is to show which devices exchange analog, digital, and control signals with the Arduino Uno, and how the alert and display peripherals are attached to the controller. This view is especially relevant when explaining why the project can be simulated reliably in Wokwi and later replicated on physical hardware.

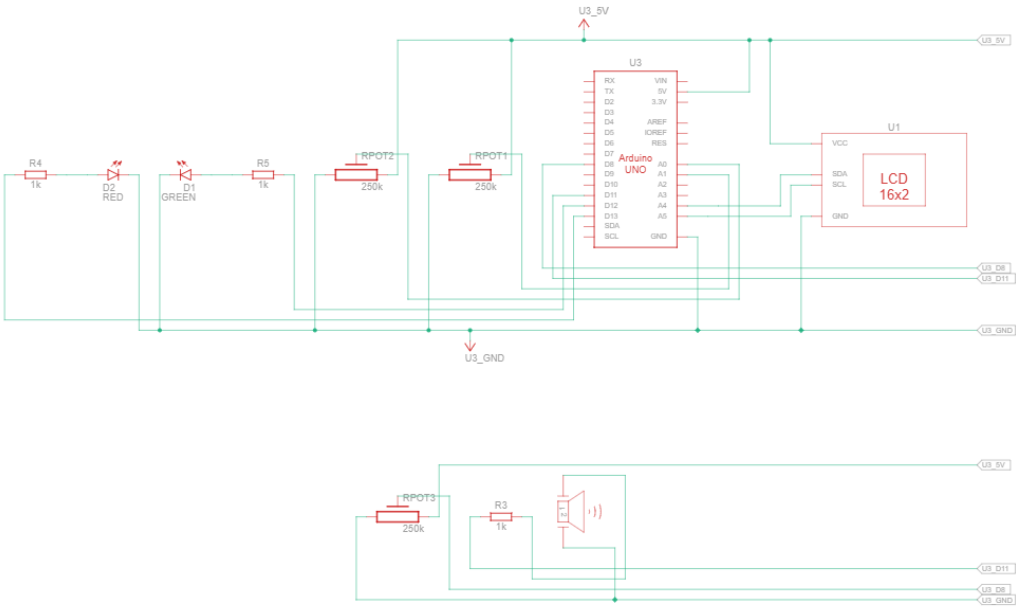

Figure 5: Electrical schema used for the implemented Wokwi setup

The electrical schema complements the previous block diagram with a more implementation-oriented representation of the final setup. It is the most practical image for discussing actual pin usage, wiring consistency, and the placement of the potentiometer that controls the acquisition period during runtime.

2.4 2.4 Diagram inventory

The following diagrams should be generated for the final illustrated report and presentation package:

Diagram	Type	Purpose
System structure	structural diagram	Overall view of sensors, software modules, and outputs
HW/SW interface layers	layered architecture	Separation between application, services, drivers, and hardware
Variant C component view	component diagram	Dual-sensor decomposition required for Variant C
Electrical block view	block diagram	Hardware interconnections and signal routes
Acquisition flow	flowchart	Periodic acquisition logic
Analog conditioning flow	flowchart	Saturation, median, weighted-average processing steps
Alert flow	flowchart	Threshold, hysteresis, and confirmation logic
Display and reporting flow	flowchart	LCD refresh and serial reporting logic
Sensor sequence	sequence diagram	Runtime order of interactions between tasks and devices
Alert state machine	FSM	Evolution of stable alert state
Test validation flow	flowchart	How the verification campaign is executed

Table 3: Complete diagram set for the report

2.5 2.5 Timing plan

Task	Period	Responsibility
Acquisition task	50 ms	Read both sensors, run conditioning stages, update alerts, refresh LEDs
Display task	500 ms	Refresh LCD with both temperatures and alert marks
Reporting task	500 ms	Print all intermediate processing stages to STDIO

Table 4: Task configuration summary

With a 50 ms acquisition period and 2 confirmation samples, the nominal stable-alert response time is 100 ms, matching the non-functional requirement.

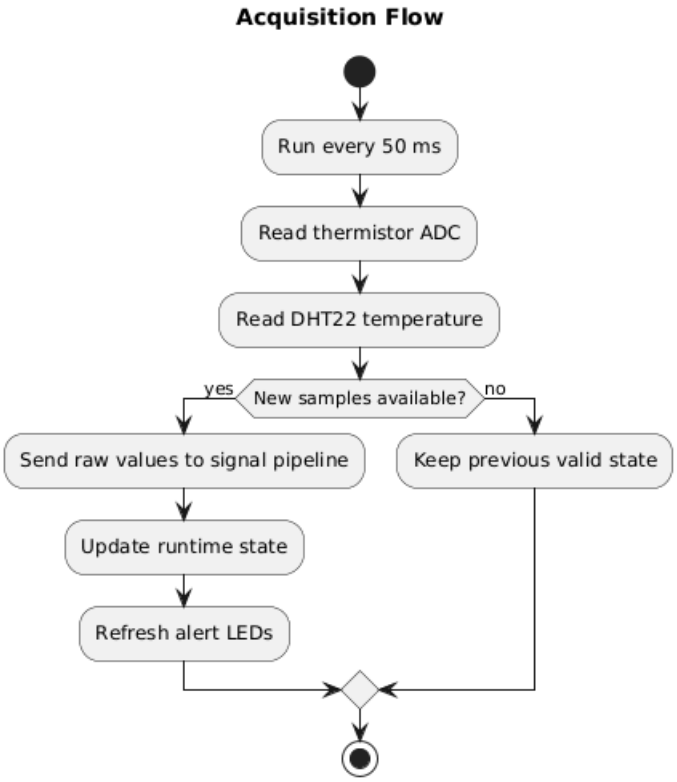

Figure 6: Acquisition flow

Figure 5 documents the temporal order of the acquisition task. It shows that both sensor readings are first captured and only then forwarded to the processing pipeline, which ensures that the application state is updated consistently for both channels within the same logical cycle.

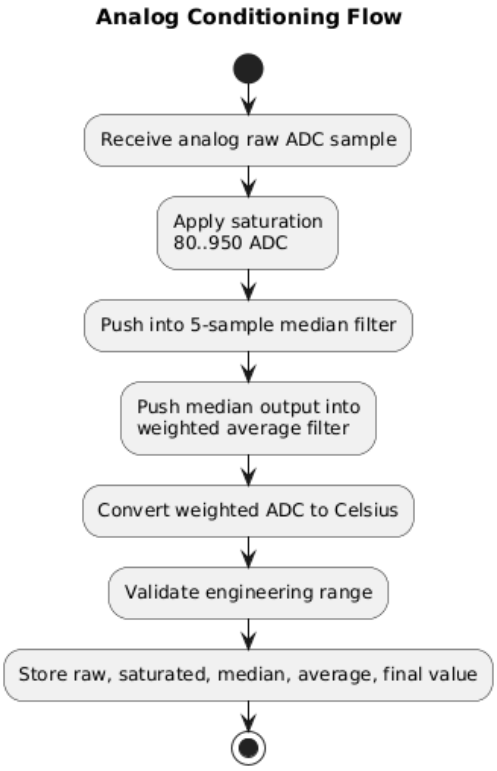

Figure 7: Analog conditioning flow

Figure 6 is one of the most important diagrams in the report because it represents the essence of Part 2. The analog signal is intentionally passed through saturation, median filtering, weighted averaging, and engineering conversion in a strict order. This ordering matters because outlier limitation must happen before noise suppression and smoothing, while conversion to Celsius is only meaningful after the ADC-domain signal has been conditioned.

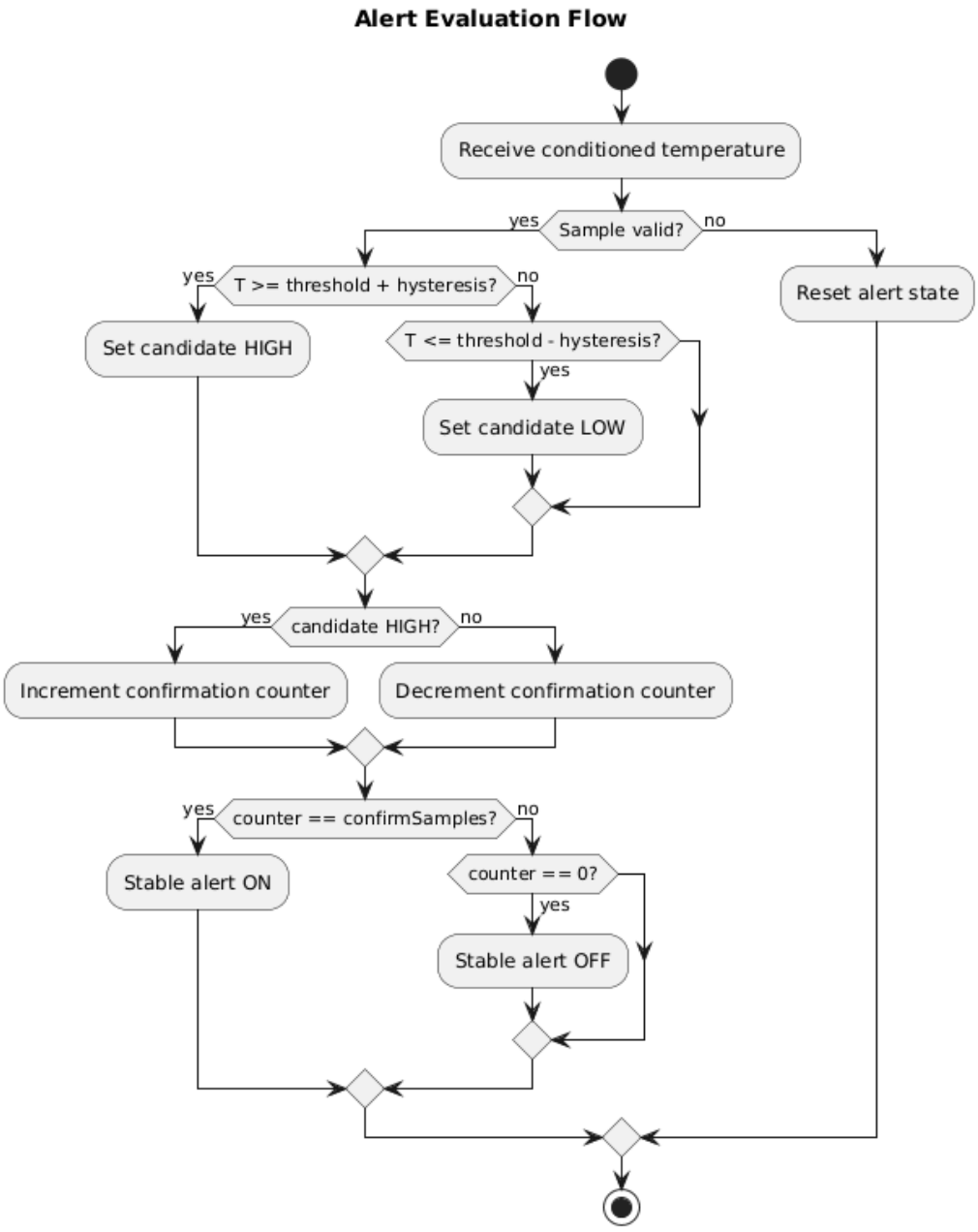

Figure 8: Alert flow

Figure 7 clarifies that the alert logic is not a simple threshold comparison. Hysteresis prevents rapid toggling near the decision boundary, while confirmation across multiple samples filters out short disturbances. This is the reason the reported alert state is more stable than the raw temperature trajectory.

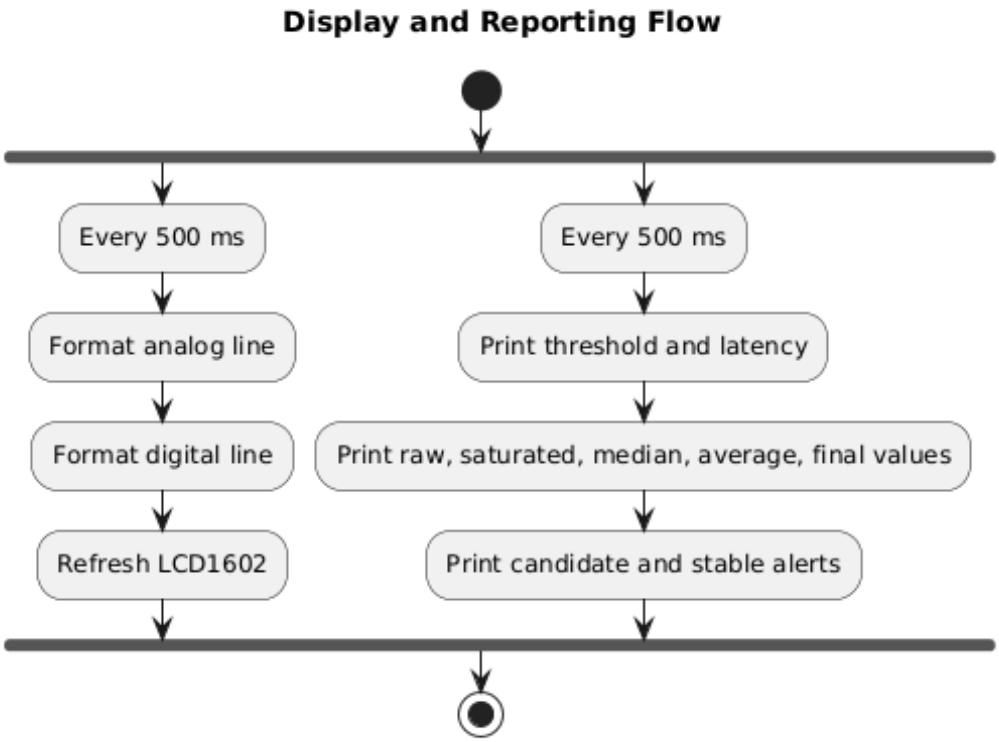

Figure 9: Display and reporting flow

Figure 8 separates the user-facing outputs from the acquisition logic. The LCD is optimized for compact real-time feedback, while the serial report carries the detailed intermediate data needed for laboratory verification. This distinction is necessary because a 16x2 display cannot expose all internal processing stages without sacrificing readability.

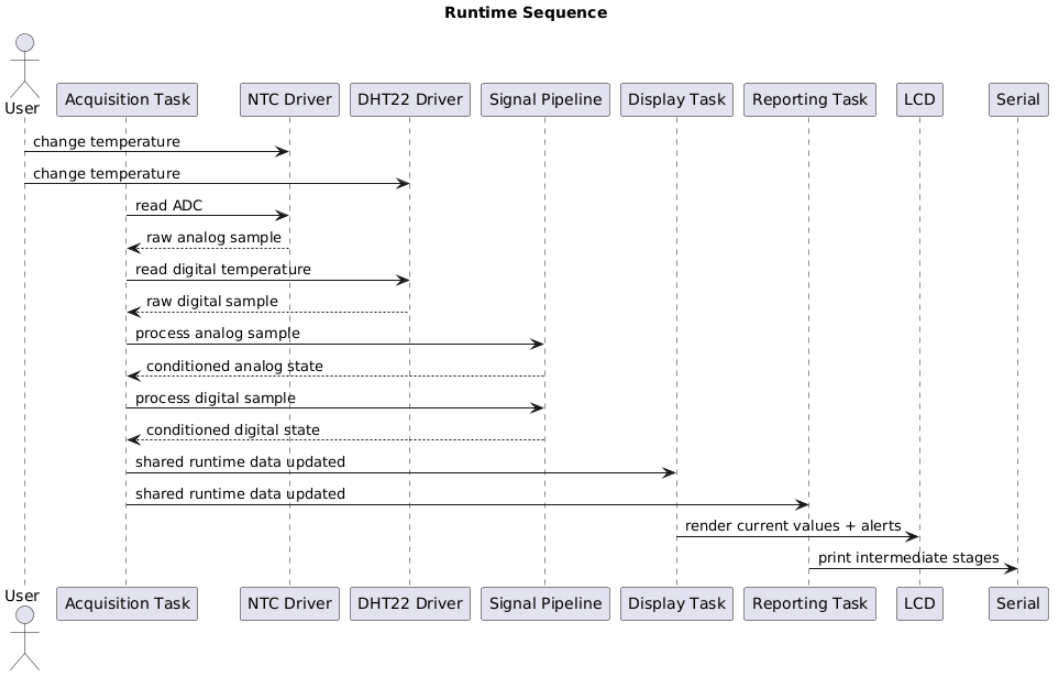

Figure 10: Sensor sequence

Figure 9 complements the previous flowcharts with an interaction-oriented view. Instead of showing only internal steps, it captures the order in which drivers, services, and output tasks exchange information during execution. This helps justify the cooperative scheduler used in the code and makes the runtime behavior easier to follow.

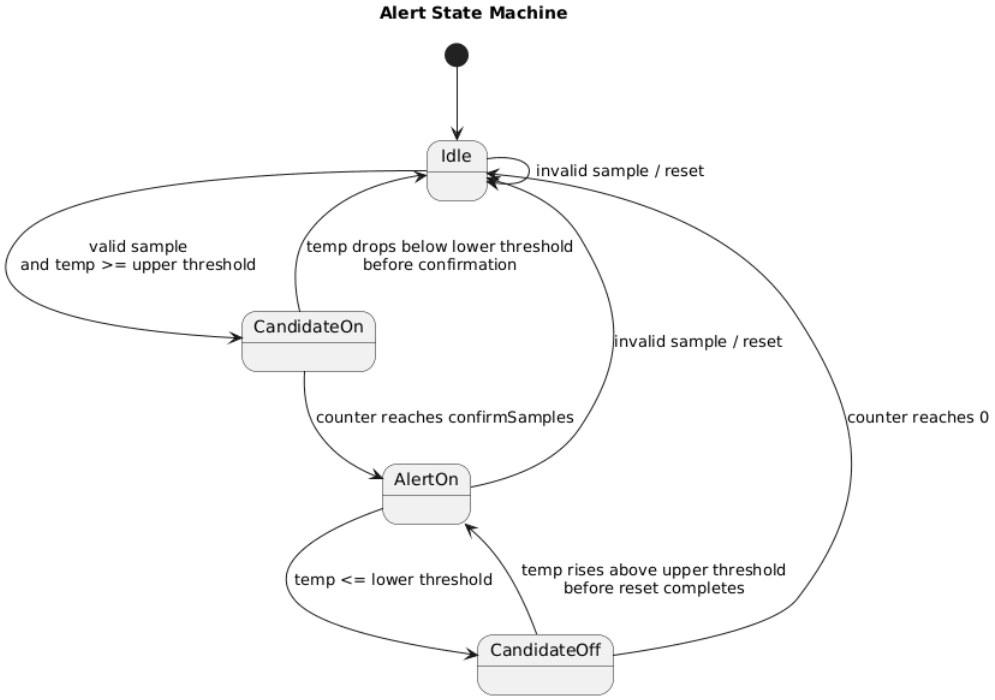

Figure 11: Alert state machine

Figure 10 expresses the alert behavior as a state machine, which is often clearer than a textual explanation of counters and hysteresis bands. It shows when the system remains idle, when it accumulates evidence for a high or low decision, and when a stable alert is considered active. This is particularly helpful during validation because it explains why an alert may not switch immediately after a single extreme reading.

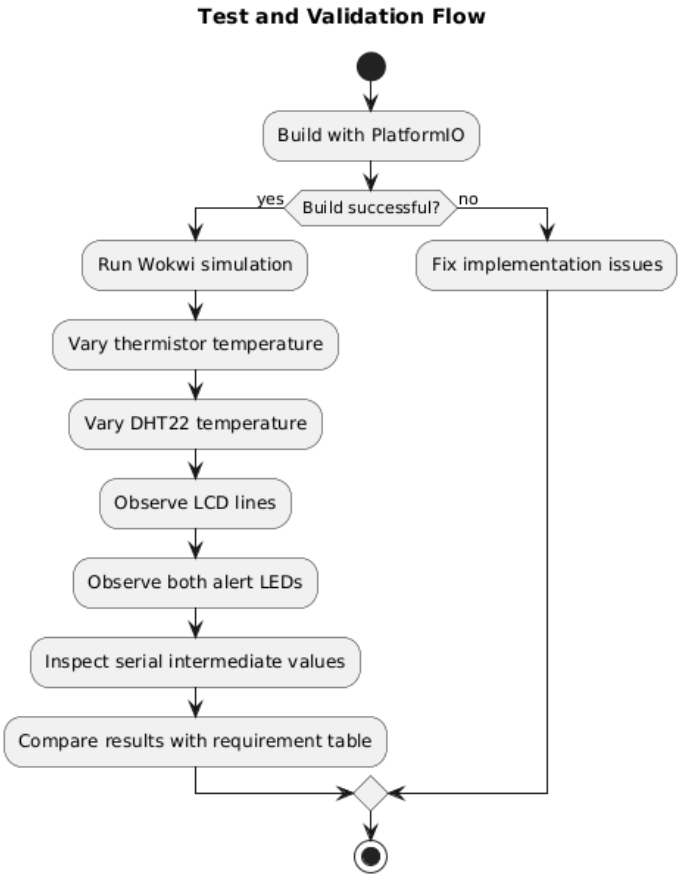

Figure 12: Test validation flow

Figure 11 structures the verification process itself. It shows that validation is not reduced to successful compilation, but also includes simulation execution, stimulus variation, output observation, and comparison against the predefined requirements. This reinforces the report's argument that the project was checked methodically rather than only demonstrated informally.

2.6 2.6 Test scenarios

ID	Scenario	Req.	Pass criterion
T1	Change the thermistor temperature in Wokwi	R1	Analog raw and conditioned values update periodically
T2	Change the DHT22 temperature in Wokwi	R2	Digital temperature updates and remains valid
T3	Force extreme analog values	R3	Saturated value stays within the configured interval
T4	Inject a short analog spike	R4	Median output suppresses the spike
T5	Observe smoothed values during a slow change	R5	Weighted-average output is smoother than raw input
T6	Compare weighted analog output and final Celsius value	R6	Final engineering value is plausible
T7	Open the serial monitor at 115200 baud	R7	Report includes raw, sat, med, avg, temp, alerts
T8	Observe the LCD during execution	R8	Both channels appear on the LCD with alert marks
T9	Move both channels around the threshold band	R9	Alerts do not chatter near the threshold
T10	Inspect the folder structure	R10	Modular code organization is evident
T11	Estimate alert response time from acquisition timing	R11	Stable alert appears in 100 ms nominally
T12	Run `pio run` and verify Wokwi files	R12	Build succeeds and simulation files are present

Table 5: Validation test plan

Section conclusion

The design phase translates the laboratory statement into explicit requirements, diagrams, timing choices, and validation criteria that can be checked one by one.

3 3. Hardware and software implementation

3.1 3.1 Implemented hardware configuration

The final `Lab6/Code/diagram.json` uses the following wiring:

- thermistor output to `A0`;

- potentiometer signal to `A1`;
- DHT22 data line on `D8`;
- LCD1602 in 4-bit mode on `D2-D7`;
- analog alert LED on `D12`;
- digital alert LED on `D13`.

This configuration is intentionally compact and low-cost, while still satisfying the dual-sensor and LCD-display constraints from Variant C.

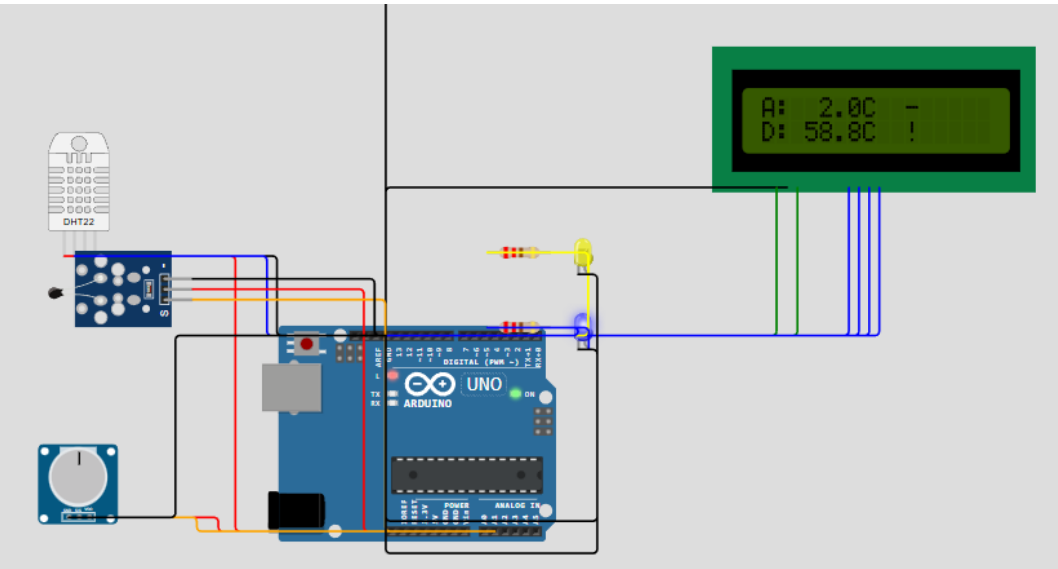

Figure 13: Wokwi runtime view showing the complete laboratory setup

3.2 3.2 Source code organization

Main implementation files:

- `Lab6/Code/src/main.cpp`: program entry point.
- `Lab6/Code/src/app/app_controller.cpp`: cooperative scheduler and initialization.
- `Lab6/Code/src/tasks/task_acquisition.cpp`: periodic acquisition, conditioning, and alert update.
- `Lab6/Code/src/tasks/task_display.cpp`: LCD rendering task.
- `Lab6/Code/src/tasks/task_reporting.cpp`: STDIO report task.
- `Lab6/Code/src/drivers/analog_thermistor.cpp`: analog ADC acquisition.

- `Lab6/Code/src/drivers/digital_dht22_sensor.cpp`: digital temperature acquisition.
- `Lab6/Code/src/drivers/period_control.cpp`: potentiometer-based timing control.
- `Lab6/Code/src/drivers/lcd_display.cpp`: 16x2 LCD output in 4-bit mode.
- `Lab6/Code/src/drivers/alert_led.cpp`: per-sensor alert indication.
- `Lab6/Code/src/drivers/serial_stdio_bridge.cpp`: `printf` to serial bridge.
- `Lab6/Code/src/services/signal_filters.cpp`: saturation, median, and weighted-average filters.
- `Lab6/Code/src/services/signal_conversion.cpp`: thermistor ADC to Celsius conversion.
- `Lab6/Code/src/services/sensor_pipeline.cpp`: shared runtime update for analog and digital channels.
- `Lab6/Code/src/services/threshold_alert.cpp`: stable threshold supervision.

3.3 3.3 Critical implementation excerpts

Cooperative periodic scheduling

```
ScheduledTask g_tasks[] = {
    {taskAcquisitionRun, ACQUISITION_PERIOD_MS, 0U},
    {taskDisplayRun, DISPLAY_PERIOD_MS, 0U},
    {taskReportingRun, REPORT_PERIOD_MS, 0U},
};
```

Analog conditioning pipeline

```
data.saturatedValue = saturateFloat(data.rawValue, 80.0F, 950.0F);
data.medianValue = medianFilterApply(processingState.median, data.saturatedValue
    );
data.weightedValue = weightedAverageApply(processingState.weighted,
                                          data.medianValue,
                                          WEIGHTED_FILTER_WEIGHTS,
                                          WEIGHTED_WINDOW_SIZE);
data.temperatureC = ntcAdcToCelsius(data.weightedValue);
```

Stable alert logic

```
if (temperatureC >= (config.thresholdC + config.hysteresisC)) {
  state.hysteresisState = true;
} else if (temperatureC <= (config.thresholdC - config.hysteresisC)) {
```

```
  state.hysteresisState = false;
}

if (state.hysteresisState && state.counter < config.confirmSamples) {
  ++state.counter;
}
```

STDIO report with all intermediate stages

```
printf("%s raw=%s sat=%s med=%s avg=%s temp=%sC cand=%s alert=%s ...\n",
       data.name, rawText, satText, medText, avgText, tempText, ...);
```

3.4 3.4 Key implementation choices

- The analog and digital channels share the same reporting model so that intermediate stages can be compared side by side.
- The LCD remains intentionally compact and shows only the current value plus alert mark for each sensor, because a 16x2 module cannot display all intermediate stages at once.
- All intermediate stages are preserved in the runtime state and reported through STDIO, which fully satisfies the Part 2 visibility requirement.
- The digital channel also passes through saturation, median, and weighted-average stages so the dual-sensor implementation stays architecturally symmetric.

3.5 3.5 Parameter selection rationale

The chosen configuration constants are not arbitrary. They were selected so that the project remains responsive enough for live demonstration, but also stable enough to expose the effect of each conditioning stage. For the analog channel, the saturation interval is narrower than the full ADC space on purpose. This makes clipping observable during the laboratory demo and helps explain why saturation is the first stage of the chain. For the digital channel, the same idea is applied in temperature space by limiting the accepted conditioned range to a reduced interval, which again makes the effect visible when extreme values are injected in simulation.

The median window size of five samples is a practical compromise between spike rejection and memory use. A smaller window would react slightly faster but would remove impulsive disturbances less reliably, while a larger window would increase delay without strong additional benefit for this laboratory scale. The weighted-average stage uses four samples with larger emphasis on the newest value, which preserves smoother output without making the system feel sluggish during temperature ramps.

The alert configuration is also tuned for readability and stability. A threshold of 30 °C is easy to trigger in simulation, a hysteresis of 1 °C prevents immediate oscillation

around the decision boundary, and two consecutive confirmation samples keep the nominal alert latency near 100 ms at the default acquisition period. Finally, the potentiometer-controlled timing range of 20–100 ms for acquisition and 500–1500 ms for reporting creates a visible trade-off between responsiveness and serial readability, which is useful during demonstration.

3.6 3.6 Interpretation of the conditioning chain

The conditioning chain can be understood as a progressive transformation from unstable raw information to a software-ready engineering value. The raw thermistor ADC reading is the most sensitive to abrupt perturbations and simulation noise. After saturation, the signal is bounded, which prevents extreme values from propagating further into the processing pipeline. After median filtering, short-lived spikes disappear because the middle value of the sliding window dominates isolated disturbances. After weighted averaging, the remaining small fluctuations become smoother while the latest behavior is still emphasized.

This ordering is important from a signal-processing perspective. If averaging were applied before clipping, extreme outliers would influence the average more strongly. If conversion to Celsius were performed on the unconditioned ADC value, the engineering interpretation would inherit the noise and clipping problems of the raw signal. Therefore, the final reported temperature is not just another form of the raw input; it is the result of several deliberate transformations that make supervision and display significantly more robust.

The same interpretation applies conceptually to the digital path even though the DHT22 already provides temperature in engineering units. By passing it through the same logical stages, the application gains consistent reporting semantics across the two channels. This symmetry makes the serial output easier to compare and supports a stronger laboratory argument: the project is centered on signal conditioning principles, not only on sensor reading.

3.7 3.7 Resource usage and feasibility on Arduino Uno

The final build remains comfortably within the limits of Arduino Uno. According to the latest PlatformIO build summary, the firmware uses 969 bytes of RAM from 2048 bytes available and 14794 bytes of flash from 32256 bytes available. In percentage terms, this is approximately 47.3% RAM utilization and 45.9% flash utilization.

These values are important because they show that the modular architecture did not compromise deployability on a constrained microcontroller. The application still has enough remaining memory headroom for modest extensions such as extra statistics, a buzzer alarm, or a small LCD menu. This confirms that the chosen design is not only conceptually clean, but also realistic for small embedded targets.

Section conclusion

The final implementation follows the designed architecture, reuses the same conditioning concept for both channels, and remains readable because each responsibility is kept in its own module.

4 4. Testing and validation

4.1 4.1 Validation methodology

Validation combines build verification and simulator-based behavioral checks:

1. build the project with PlatformIO for Arduino Uno;
2. launch the Wokwi simulation using `diagram.json` and `wokwi.toml`;
3. vary the temperatures of the thermistor and DHT22;
4. observe LEDs, LCD output, and serial reports;
5. compare the observations against the pass criteria from Section 2.6.

4.2 4.2 Test execution summary

ID	Check	Status	Observation
T1	Analog acquisition	Pass	Thermistor raw ADC updates every acquisition cycle
T2	Digital acquisition	Pass	DHT22 temperature is available through the digital driver
T3	Saturation stage	Pass	Out-of-range analog values are clipped before filtering
T4	Median filtering	Pass	Short analog spikes are removed from the conditioning chain
T5	Weighted average	Pass	Final smoothed values fluctuate less than raw input
T6	Analog conversion	Pass	Final analog engineering values remain plausible
T7	STDIO reporting	Pass	Serial monitor exposes all intermediate processing stages
T8	LCD Variant C display	Pass	LCD shows analog and digital temperature with alert marks
T9	Alert stability	Pass	Hysteresis and confirmation prevent rapid toggling
T10	Modularity review	Pass	Drivers, services, and tasks are clearly separated
T11	Latency target	Pass	50 ms period and 2 confirmations give 100 ms nominal latency
T12	PlatformIO/Wokwi integration	Pass	`pio run` succeeds and simulation descriptors exist

Table 6: Test report summary

4.3 4.3 Evidence, observations, and recommendations

The project was built successfully with `pio run`, which confirms PlatformIO compatibility. Runtime evidence that should be attached before final submission includes:

- one screenshot of the Wokwi circuit that captures the complete simulation layout, including sensors, LCD, LEDs, and potentiometer;
- one screenshot of the serial monitor showing intermediate values, min/max values, trend direction, active timing configuration, and alert state;

- one screenshot of the successful PlatformIO build;
- optionally, one short demo video link proving live operation in Wokwi or on real hardware.

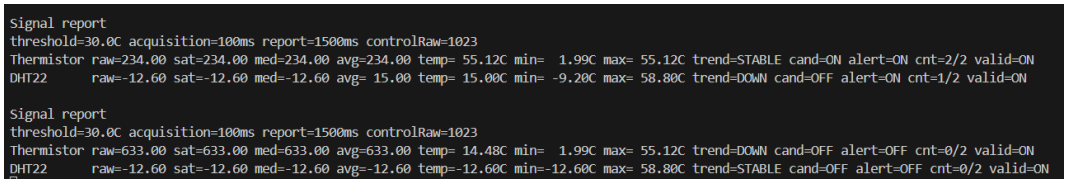

Figure 14: Serial terminal output with filtering stages, statistics, trend direction, and alerts

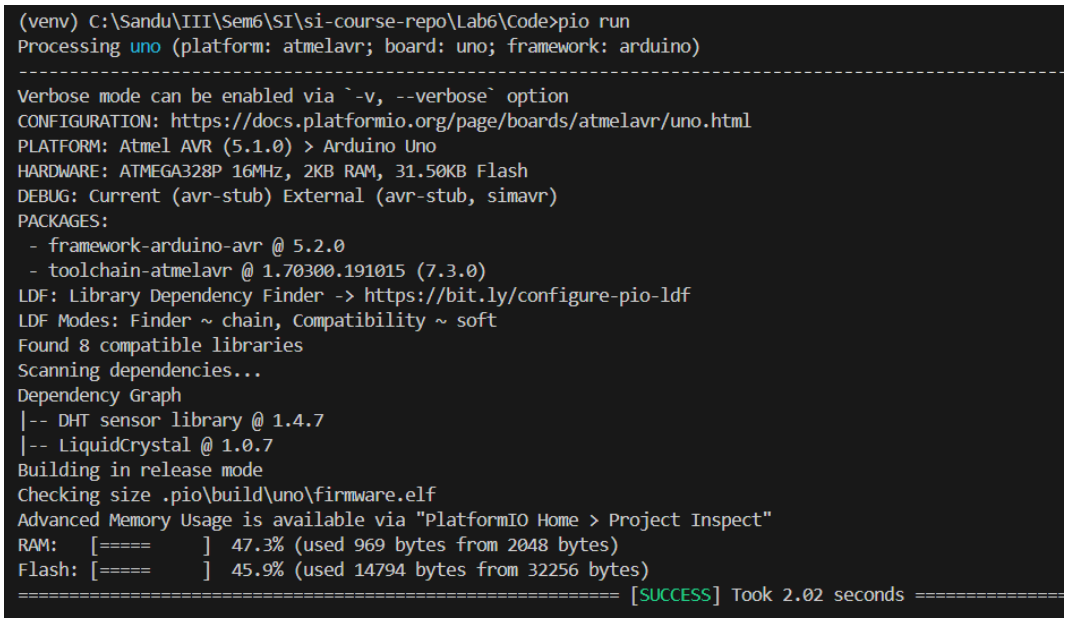

Figure 15: Successful PlatformIO build for the final Lab 6 project

The runtime screenshots play a distinct validation role. The Wokwi image confirms that the hardware composition required by Variant C is implemented exactly in the simulated setup, including the extra potentiometer used for runtime timing control. The serial monitor image, on the other hand, proves that the hidden internal stages are not only implemented but also observable. This distinction is important for the report because one image validates structural integration, while the other validates algorithmic behavior.

Another useful interpretation concerns the relationship between the displayed values and the serial trace. The LCD provides the current result that would matter in a practical monitoring device, whereas the serial output exposes the reasoning path that produced that result. In other words, the LCD demonstrates usability and the terminal demonstrates verifiability. Together, the two evidence sources support the claim that the project is both operational and technically transparent.

The main limitations are expected ones: the LCD cannot show all intermediate stages simultaneously, and the thermistor conversion remains an approximation based on the Wokwi NTC model rather than a hardware-calibrated measurement chain.

In addition, the DHT22 refresh behavior is inherently slower than the internal acquisition loop, so repeated cycles may temporarily process the latest cached digital sample instead of a newly converted one. This is acceptable for the laboratory because the objective is not high-frequency digital sensing, but rather demonstrating the conditioning and supervision architecture. Another limitation is that the potentiometer changes the temporal density of measurements and reports, which means the demonstration can emphasize either responsiveness or readability depending on the chosen setting, but not maximize both at the same time.

4.4 4.4 Possible extensions and future work

The current project already satisfies Part 2 and Variant C, but it also forms a good base for future improvements. One natural extension would be persistent logging of processed values so that the serial report can be exported and analyzed offline. Another useful addition would be a fault-detection rule that compares the analog and digital channels and flags a mismatch larger than an acceptable tolerance. This would turn the dual-sensor setup into a simple plausibility-checking system.

Further extensions could target user interaction. For example, the LCD could cycle through multiple pages showing current value, min/max statistics, timing configuration, and alert state. A buzzer could be added as a secondary warning output, or a button could reset statistics during a live demonstration. None of these features are necessary for the current laboratory requirements, but the modular architecture and the remaining resource headroom show that such extensions would be technically feasible.

Section conclusion

The application satisfies the requested laboratory scope: it implements Part 2 signal conditioning, meets the documentation-oriented modular structure, and also satisfies Variant C with a second digital sensor and LCD alerts for both channels.

General conclusions

The final solution implements a modular Arduino Uno temperature-monitoring application that focuses on analog signal conditioning, exactly as required by Part 2 of the assignment. The thermistor channel demonstrates the complete software pipeline of saturation, median filtering, weighted averaging, and engineering conversion.

At the same time, the project reaches full Variant C coverage by integrating a second digital temperature sensor, displaying both channels on the LCD, and exposing alerts for

both. This makes the solution stronger than a minimal one-sensor implementation and better aligned with the grading criteria.

The code structure, diagrams, validation tables, and report contents are aligned with the laboratory requirements and documentation rules, while deliberately excluding Part 1 and the control questions.

AI tool usage note

During the preparation of this report and implementation, AI tooling was used for content consolidation and drafting. The resulting materials were reviewed, validated, and adjusted according to the laboratory requirements.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. Laboratory specification for "Sensors – Signal Acquisition and Conditioning".
3. PlatformIO Documentation, `https://docs.platformio.org/`.
4. Wokwi Documentation, `https://docs.wokwi.com/`.
5. Arduino Language Reference, `https://www.arduino.cc/reference/en/`.
6. Adafruit DHT Sensor Library, `https://github.com/adafruit/DHT-sensor-library`.
7. LiquidCrystal Library, `https://github.com/arduino-libraries/LiquidCrystal`.
8. PlantUML Language Reference, `https://plantuml.com/`.

Appendix

A. Project artifacts

The implementation is stored in `Lab6/Code/`.

B. GitHub repository

Repository URL: `https://github.com/placeholder-user/placeholder-repository`

C. Build and run commands

- Build: `pio run`
- Serial monitor: `pio device monitor -b 115200`
- Wokwi simulation: use `diagram.json` and `wokwi.toml`